REPORTE DE PRÁCTICA NO. 7

Construcción de un Analizador Léxico y Sintáctico con Lex y Yacc

ALUMNOS:

Arrazola Ibarra Juan Pablo
Vazquez Pontaza Fernando
Olivares Roque Diego
Caballero Rossano Gabriel

Dr. Eduardo Cornejo-Velázquez

Introducción

El proceso de compilación de un lenguaje de programación se divide en varias fases, siendo el análisis léxico y el análisis sintáctico los pilares fundamentales para la comprensión del código fuente. En esta práctica, se desarrolló un analizador capaz de reconocer y validar un subconjunto de instrucciones basadas en la sintaxis de Python. Utilizando las herramientas Lex (Flex) y Yacc (Bison), se construyó un programa que no solo tokeniza cadenas de texto, sino que evalúa jerárquicamente estructuras de control de flujo como `if`, `while`, `for`, operaciones aritméticas y declaraciones de variables, aplicando mecanismos de recuperación de errores para garantizar un análisis continuo.

Marco Teórico

Análisis Léxico (Lex/Flex)

El análisis léxico es la primera fase de un compilador. Su función es leer los caracteres del código fuente y agruparlos en unidades con significado llamadas tokens. Estas unidades representan palabras reservadas, identificadores, números, operadores y símbolos especiales.
Lex o Flex permite generar analizadores léxicos utilizando expresiones regulares y autómatas finitos deterministas.

Análisis Sintáctico (Yacc/Bison)

El análisis sintáctico recibe los tokens generados por el analizador léxico y verifica que estén organizados conforme a las reglas gramaticales del lenguaje.
Yacc o Bison genera analizadores sintácticos de tipo LALR(1), capaces de validar estructuras complejas mediante gramáticas libres de contexto.

Gramáticas Libres de Contexto

Las Gramáticas Libres de Contexto son reglas formales utilizadas para describir la sintaxis de los lenguajes de programación. Estas gramáticas permiten representar estructuras anidadas y recursivas, como condiciones y ciclos.

Notación BNF

La notación BNF (Backus-Naur Form) es una forma estándar de representar reglas gramaticales. Facilita la definición formal de la sintaxis de un lenguaje.

Objetivo General

Implementar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar la estructura condicional `if` capaz de realizar la detección de errores y la validación de gramáticas en un lenguaje de programación de alto nivel.

Objetivos Específicos

- Emplear las herramientas conceptuales (autómatas y gramáticas libres de contexto) para definir cómo lograr el objetivo de la práctica.
- Diseñar un programa en Lex que identifique los tokens asociados a palabras reservadas como `if`, `while`, `for`, expresiones condicionales y bloques de código.
- Implementar un programa en Yacc que verifique la correcta construcción de estructuras condicionales mediante una gramática formal.
- Detectar errores sintácticos y continuar el análisis del código fuente.

Desarrollo

Para llevar a cabo esta práctica se desarrolló un analizador léxico y sintáctico utilizando las herramientas Lex y Yacc.
En primer lugar, se definieron los autómatas necesarios para reconocer los diferentes tokens requeridos en la práctica. Estos tokens incluyen palabras reservadas, identificadores, números, operadores aritméticos y operadores relacionales.
Posteriormente, se utilizó la notación BNF para describir formalmente la gramática del lenguaje implementado.

Gramática utilizada

```
<codigo> ::= <linea> | <codigo> <linea>

<linea> ::= <estructura> SALTO
          | SALTO
          | error SALTO

<estructura> ::= <declaracion>
               | <sent_if>
               | <sent_while>
               | <sent_for>
               | <sent_print>

<sent_if> ::= IF <condicion> ":" <estructura>
            | IF <condicion> ":" SALTO <estructura>

<sent_while> ::= WHILE <condicion> ":" <estructura>

<sent_for> ::= FOR ID IN ID ":" <estructura>

<declaracion> ::= ID "=" <expresion>

<expresion> ::= ID
              | NUM_INT
              | NUM_FLOAT
              | CADENA
              | <expresion> <operador_aritmetico> <expresion>

<condicion> ::= <expresion> <operador_relacional> <expresion>
```

Funcionamiento de Lex

El archivo `analizador.l` se encargó de reconocer y clasificar los diferentes tokens utilizados dentro del lenguaje implementado. Para ello, se utilizaron expresiones regulares que permitieron identificar palabras reservadas, identificadores, números, operadores y símbolos especiales.
Entre las palabras reservadas reconocidas se encuentran:

- if
- while
- for
- print

- in

Además, el analizador léxico identificó correctamente:

- Números enteros.
- Números flotantes.
- Identificadores.
- Operadores aritméticos.
- Operadores relacionales.
- Cadenas de texto.
- Paréntesis y símbolos especiales.

También se implementó el manejo de espacios, tabulaciones, comentarios y saltos de línea para evitar errores durante el análisis.
El código implementado en el archivo `analizador.l` fue el siguiente:

```
%option noyywrap
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "y.tab.h"

void yyerror(char *s);
%}

%%
"if"                    { return IF; }
"for"                   { return FOR; }
"while"                 { return WHILE; }
"print"                 { return PRINT; }
"in"                    { return IN; }
":"                     { return DOS_PUNTOS; }
"("                     { return LPAREN; }
")"                     { return RPAREN; }

[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
[0-9]+"."[0-9]+         { return NUM_FLOAT; }
[0-9]+                  { return NUM_INT; }

"<"|">"|"=="|"!="|"<="|">="   { return OPERADOR_RELACIONAL; }
"="                     { return IGUAL; }
"+"                     { return MAS; }
"-"                     { return MENOS; }
"*"                     { return POR; }
"/"                     { return DIV; }

"\""[^\"]*"\""          { return CADENA; }

"#".*  ;
"\"\"\""([^\"]|"\""[^\"]|"\"\""[^\"])*"\"\"\""  ;
```

```
[ \t]                   ;
\n                      { return SALTO; }
.                       { printf("Error lexico: %s\n", yytext); }
%%
```

Funcionamiento de Yacc

El archivo `sintactico.y` recibió los tokens generados por el analizador léxico y verificó que cumplieran con las reglas definidas en la gramática.
Se implementó un análisis Bottom-Up o Shift-Reduce, donde primero se reducen las expresiones internas y posteriormente las estructuras externas.
La gramática desarrollada permitió reconocer:

- Declaraciones de variables.
- Expresiones aritméticas.
- Condiciones relacionales.
- Estructuras if.
- Ciclos while.
- Ciclos for.
- Funciones print.

Además, se implementó recuperación de errores utilizando:

```
linea: error SALTO { yyerrok; }
```

Esto permitió continuar el análisis incluso después de detectar errores sintácticos.
El código implementado en el archivo `sintactico.y` fue el siguiente:

```
%{
#include <stdio.h>
#include <stdlib.h>

extern FILE *yyin;
int yylex();
void yyerror(char *s);
%}

%token IF FOR WHILE PRINT DOS_PUNTOS ID NUM_INT NUM_FLOAT
%token OPERADOR_RELACIONAL IGUAL SALTO LPAREN RPAREN IN CADENA
%token MAS MENOS POR DIV

%left MAS MENOS
%left POR DIV

%%
codigo:
    linea
    | codigo linea
    ;

linea:
```

```
    estructura SALTO {
        if ($1 == 1)
            printf("ASIGNACION/DECLARACION-COMPLEJA\n");
        else if ($1 == 2)
            printf("ESTRUCTURA-IF-\n");
        else if ($1 == 3)
            printf("ESTRUCTURA-WHILE-\n");
        else if ($1 == 4)
            printf("ESTRUCTURA-FOR-\n");
        else if ($1 == 5)
            printf("FUNCION-PRINT\n");
    }
    | SALTO
    | error SALTO { yyerrok; }
    ;

estructura:
    declaracion   { $$ = 1; }
    | sent_if     { $$ = 2; }
    | sent_while  { $$ = 3; }
    | sent_for    { $$ = 4; }
    | sent_print  { $$ = 5; }
    ;

sent_if:
    IF condicion DOS_PUNTOS estructura
    | IF condicion DOS_PUNTOS SALTO estructura
    ;

sent_while:
    WHILE condicion DOS_PUNTOS estructura
    | WHILE condicion DOS_PUNTOS SALTO estructura
    ;

sent_for:
    FOR ID IN ID DOS_PUNTOS estructura
    | FOR ID IN ID DOS_PUNTOS SALTO estructura
    ;

declaracion:
    ID IGUAL expresion
    ;

expresion:
    ID | NUM_INT | NUM_FLOAT | CADENA
    | expresion MAS expresion
    | expresion MENOS expresion
    | expresion POR expresion
    | expresion DIV expresion
    | LPAREN expresion RPAREN
    ;

sent_print:
    PRINT LPAREN contenido_print RPAREN
```

```
    ;

contenido_print:
    expresion | /* vac o */
    ;

condicion:
    expresion OPERADOR_RELACIONAL expresion
    ;

%%

void yyerror(char *s) {
    fprintf(stderr,
    ">>> ERROR SINTACTICO: Estructura no valida <<<\n");
}

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *archivo = fopen(argv[1], "r");
        if (!archivo) {
            printf("Error al abrir el archivo.\n");
            return 1;
        }
        yyin = archivo;
    }

    printf("+++ Analizando Codigo Python +++\n");
    yyparse();
    printf("+++ Analisis Finalizado +++\n");

    return 0;
}
```

Archivo de prueba utilizado

Para comprobar el correcto funcionamiento del analizador léxico y sintáctico, se creó un archivo de prueba que contiene diferentes estructuras válidas e inválidas inspiradas en la sintaxis de Python.
El objetivo de este archivo fue verificar que el programa pudiera:

- Reconocer declaraciones de variables.
- Validar operaciones aritméticas.
- Detectar estructuras `if`, `while` y `for`.
- Procesar funciones `print`.
- Detectar errores sintácticos.
- Continuar el análisis después de encontrar errores.

El archivo utilizado durante las pruebas fue el siguiente:

```
x = 10
y = 20 + 5
```

```
mensaje = "Hola Mundo"

if x > 5:
    print("Valor valido")

while x < 100:
    x = x + 1

for i in numeros:
    print(i)

print("Analisis completado")

do while x > 0:

variable_final = 100
```

En el ejemplo anterior se incluyeron instrucciones válidas y una estructura inválida (`do while`) para comprobar el mecanismo de recuperación de errores implementado en Yacc.

Proceso de compilación con Lex y Yacc

Para la ejecución del analizador léxico y sintáctico se siguieron los siguientes pasos:

1. Se creó el archivo del analizador léxico `lexico.l`.
2. Se ejecutó el comando:

   ```
   lex analizador.l
   ```

3. Se creó el archivo del analizador sintáctico `sintactico.y`.
4. Se ejecutó el comando:

   ```
   yacc -dy sintactico.y
   ```

5. Se compiló todo utilizando:

   ```
   cc lex.yy.c y.tab.c -o compilador
   ```

6. Finalmente, se ejecutó el programa para validar las estructuras definidas.

Resultados

Durante la ejecución del programa se realizaron múltiples pruebas utilizando declaraciones de variables, operaciones aritméticas y estructuras de control inspiradas en la sintaxis de Python.
El analizador desarrollado fue capaz de:

- Reconocer correctamente palabras reservadas como `if`, `while`, `for` y `print`.
- Validar expresiones aritméticas y condiciones relacionales.
- Detectar estructuras válidas dentro del código fuente.
- Procesar estructuras anidadas correctamente.
- Detectar errores sintácticos sin detener completamente la ejecución.
- Continuar el análisis después de encontrar errores gracias al mecanismo `yyerrok`.

Se realizaron pruebas con instrucciones válidas e inválidas para comprobar el comportamiento del compilador. Por ejemplo, al ingresar estructuras válidas como ciclos `while`, ciclos `for` y estructuras `if`, el analizador imprimió mensajes indicando que la estructura fue reconocida correctamente.
También se probaron líneas inválidas utilizando sintaxis incorrecta. En esos casos, el programa detectó el error y mostró el mensaje:

```
>>> ERROR SINTACTICO: Estructura no valida <<<
```

A pesar de encontrar errores, el sistema continuó procesando las siguientes instrucciones, demostrando un funcionamiento robusto y estable.
La siguiente imagen muestra el resultado final obtenido durante la ejecución del programa:
En la imagen anterior se observa cómo el programa reconoce correctamente múltiples estructuras válidas como:

- ESTRUCTURA WHILE
- ESTRUCTURA FOR
- ESTRUCTURA IF
- FUNCION PRINT

Posteriormente, también se muestran varios errores sintácticos detectados por el analizador, los cuales fueron manejados adecuadamente sin provocar el cierre del programa.
Finalmente, el mensaje:

```
+++ Analisis Finalizado +++
```

indica que el proceso terminó correctamente después de analizar todas las líneas del archivo de entrada.
Estos resultados demuestran que el analizador léxico y sintáctico construido con Lex y Yacc funciona correctamente y cumple con los objetivos planteados en la práctica.

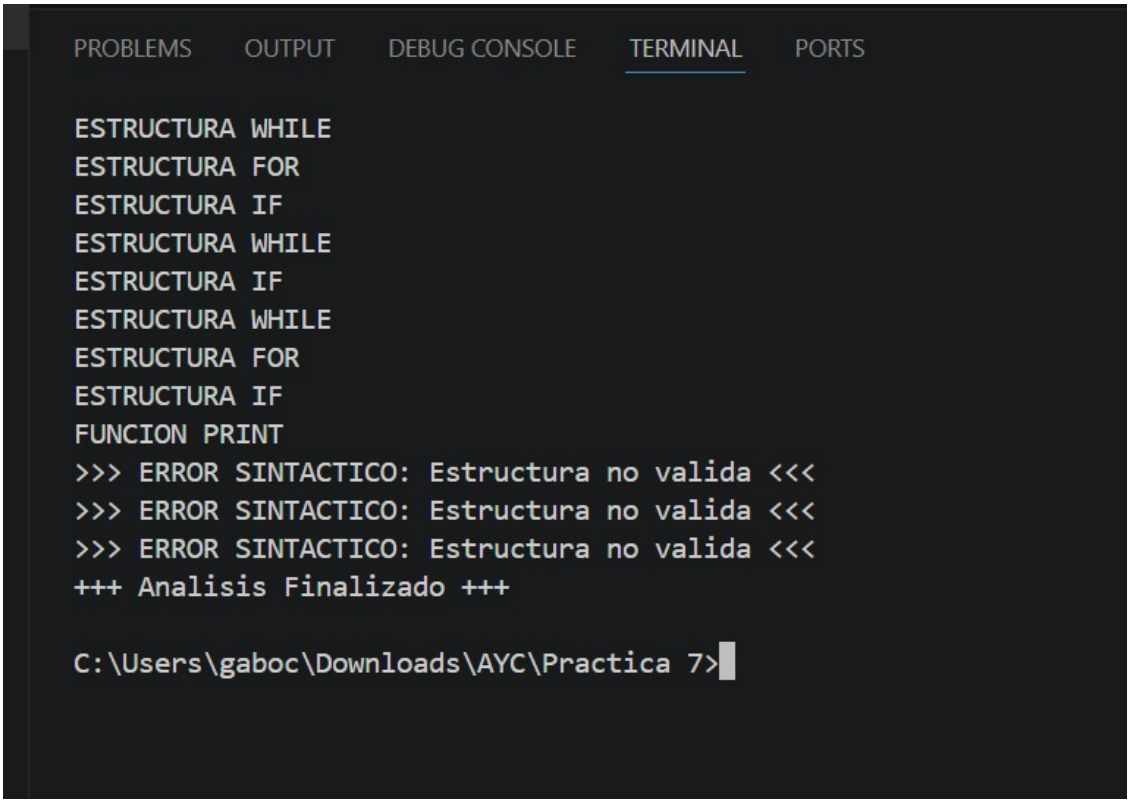

Figure 1: Resultado final del analizador sintáctico mostrando estructuras válidas y detección de errores.

Conclusiones

A lo largo del desarrollo de esta práctica se logró comprender de manera más profunda el funcionamiento de las primeras fases de un compilador, específicamente el análisis léxico y el análisis sintáctico. Mediante el uso de las herramientas Lex y Yacc fue posible implementar un sistema capaz de reconocer y validar instrucciones inspiradas en la sintaxis del lenguaje Python, permitiendo observar de forma práctica cómo trabajan los compiladores al interpretar código fuente.

Durante la implementación del analizador léxico se comprobó la importancia de las expresiones regulares y de los autómatas finitos para el reconocimiento de tokens. Gracias a Lex fue posible identificar correctamente palabras reservadas, identificadores, números enteros, números flotantes, cadenas de texto y operadores aritméticos y relacionales. Esto permitió transformar el código fuente en una secuencia estructurada de tokens que posteriormente fue utilizada por el analizador sintáctico.

En la etapa de análisis sintáctico se logró validar la correcta construcción de estructuras como declaraciones de variables, expresiones aritméticas, condiciones y ciclos. El uso de gramáticas libres de contexto permitió definir formalmente las reglas del lenguaje y comprobar cómo Yacc construye el árbol sintáctico mediante un análisis Bottom-Up o Shift-Reduce. Gracias a este tipo de análisis, el sistema pudo reducir primero las expresiones más internas y posteriormente validar las estructuras externas, respetando la jerarquía de las instrucciones.

Uno de los aspectos más importantes observados durante la práctica fue el manejo de estructuras anidadas. El analizador fue capaz de reconocer correctamente estructuras `if`, `while` y `for` dentro de otras estructuras, demostrando la capacidad de las gramáticas libres de contexto para modelar comportamientos recursivos y complejos dentro de un lenguaje de programación.

Asimismo, se comprobó la importancia de implementar mecanismos de recuperación de errores sintácticos. Gracias al uso de la regla:

```
linea: error SALTO { yyerrok; }
```

el analizador pudo detectar instrucciones inválidas sin finalizar abruptamente la ejecución. Esto permitió continuar el análisis del resto del archivo, mostrando mensajes de error claros y manteniendo el funcionamiento estable del programa. Este tipo de recuperación es muy importante en compiladores reales, ya que permite proporcionar retroalimentación útil al usuario incluso cuando existen errores en el código fuente.

Otro aspecto relevante fue observar cómo las herramientas Lex y Yacc facilitan considerablemente el desarrollo de analizadores y compiladores. Sin estas herramientas, la implementación manual de un analizador léxico y sintáctico sería mucho más compleja y propensa a errores. Lex simplifica el reconocimiento de patrones mediante expresiones regulares, mientras que Yacc automatiza la construcción de analizadores sintácticos utilizando reglas gramaticales.

Además, la práctica permitió reforzar conceptos teóricos vistos durante el curso, como gramáticas libres de contexto, notación BNF, autómatas, derivaciones, árboles sintácticos y análisis ascendente. Al aplicar estos conceptos en un proyecto funcional, fue posible comprender mejor su utilidad dentro del área de compiladores y procesamiento de lenguajes.

Finalmente, se concluye que la construcción de analizadores léxicos y sintácticos representa una parte esencial en el desarrollo de compiladores, intérpretes y procesadores de lenguaje. La práctica demostró que mediante una correcta definición de tokens y gramáticas es posible construir sistemas capaces de validar estructuras complejas, detectar errores y procesar instrucciones de manera eficiente. Asimismo, se comprobó que Lex y Yacc son herramientas poderosas y fundamentales para el diseño de compiladores modernos y sistemas de análisis de código fuente.

Bibliografía

References

[1] Aho, A. V., Sethi, R., Ullman, J. D. (1998). *Compiladores: Principios, técnicas y herramientas*. Addison Wesley Longman.

[2] Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega.

[3] Levine, J., Mason, T., Brown, D. (1992). *Lex & Yacc*. O'Reilly Media.